

TEORÍA (8 puntos)**Apellidos y nombre:****Pregunta 1.** ¿Qué es la «programación en pareja» («*pair programming*»)?**Pregunta 2.** Enumera tres técnicas de depuración por fuerza bruta**Pregunta 3. Técnicas basadas en puntos de decisión.**

Dado el código fuente que se muestra a continuación, define un juego de pruebas utilizando la técnica de cobertura de condiciones y decisiones modificada. Justifica tu respuesta.

```
if mes >= 1 and mes <= 12
    then mostrarCalendario(mes);
end if;
```

Pregunta 4. Describe tres métricas que permitan informar de forma específica de la efectividad de las pruebas

Pregunta 5. Describe brevemente cómo se calculan los puntos de prueba iniciales utilizados como base para aplicar la técnica de estimación denominada “Análisis de Puntos de Prueba”.

Pregunta 6. ¿En qué consiste la base de las pruebas (*test basis*)? Enumera tres elementos que se pueden utilizar como base de las pruebas.

Pregunta 7. Describe brevemente la técnica de diseño de pruebas denominada *scenario testing* y nombra alguna herramienta que permita automatizar de forma directa las pruebas diseñadas con esta técnica.

Pregunta 8. ¿Qué se puede automatizar cuando se realizan pruebas de accesibilidad? Nombra alguna herramienta que conozcas para realizar pruebas de accesibilidad sobre aplicaciones Android.

Pregunta 9. Distingue las posibles formas de ejecutar las pruebas.

Pregunta 10. Enumera tres tipos distintos de objetivos de pruebas que se pueden utilizar como punto de partida para realizar el análisis de riesgos de un producto y justifica su interés.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

Diseña pruebas para el siguiente código utilizando la técnica de pruebas de caminos con **nivel de profundidad 2**:

1. Obtén primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/expresiones y nodos que se indica en el propio código (0,2 puntos).
2. Los subcaminos FGIBJ y FHBCJ son imposibles. Explica la razón (0,2 puntos).
3. Aplica la técnica de pruebas de caminos con profundidad 2 para obtener el conjunto de caminos completos correspondiente (0,7 puntos). Al hacerlo, ten en cuenta la existencia de los dos subcaminos imposibles mencionados anteriormente. El primero de ellos, si el grafo está bien hecho, tiene profundidad 2, mientras que el segundo tiene profundidad 3.
4. Finalmente, establece casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,9 puntos).

```
/**
 * Dada una palabra oculta, una secuencia de letras correspondiente a
 * intentos de adivinar letras de la palabra y un número máximo de fallos
 * permitidos, determina si dicha secuencia permite adivinar todas las
 * letras de la palabra oculta en una partida del juego del ahorcado.
 * @param palabraOculta palabra que hay que adivinar en la partida
 * @param Letras vector correspondiente a los intentos del jugador por
 *             adivinar las letras de la palabra
 * @param maxFallos número de fallos que hace que se pierda la partida
 * @return true si la secuencia de letras permite descubrir la palabra
 *         sin alcanzar el número máximo de fallos, false en caso
 *         contrario (se alcanza el número máximo de fallos o quedan
 *         letras de la palabra oculta sin acertar)
 */
public boolean jugar(String palabraOculta, char[] letras, int maxFallos) {
    A int fallos = 0;
    int aciertos = 0;
    int i = 0;
    while ( B fallos < maxFallos
            && C aciertos < palabraOculta.length()
            && D i < letras.length ) {
        E final char letra = letras[i];
        if ( F palabraOculta.contains(String.valueOf(letra)) ) {
            G // Se cuenta el número de repeticiones de «letra» en la palabra
              // y se incrementa «aciertos»:
            aciertos += (int) palabraOculta.chars()
                .filter(ch -> ch == letra).count();
            // Se elimina de la palabra la letra acertada:
            palabraOculta = palabraOculta.replace(letra, '-');
        } else {
            H fallos++;
        }
        I i++;
    }
    J return aciertos == palabraOculta.length();
}
```